PARCIAL 3

INTEGRANTES

DAVID SANTIAGO GUERRERO GARCIA 01250371065

NIKOL FERNANDA MEDINA MONTERREY 01250371052

PROFESOR

CARLOS ALBERTO CARRASCAL AVENDAÑO

UNIVERSIDAD DE SANTANDER

PROGRAMACION DE COMPUTADORES I

BUCARAMANGA SANTANDER

8 NOVIEMBRE 2026

ANALISIS

DATOS DE ENTRADA	PROCESOS	DATOS DE SALIDA
Datos del Conglomerado y Ubicación	Organización territorial y brigadas	Resultados de consulta en pantalla (Consola)
ID o número del conglomerado	Desplazamiento y geolocalización	Indicadores e informes de especies
Verificación geolocalización ubicación de las subparcelas	Conteo de árboles	Archivos de texto plano
Datos de la Brigada	Toma y registro de muestras	
Nombre o identificación de la brigada de levantamiento	Análisis de laboratorio	
Datos de Conteo (según el diámetro de los árboles)	Procesamiento de datos y cálculo de indicadores	
Cantidad de árboles fustales		
Cantidad de árboles latizales		
Cantidad de árboles brinzales		
Datos de Muestreo (para la toma de muestras)		
Nombre común de la especie		
Uso de la especie (*medicinal, alimenticio o maderable*)		
Código de identificación asignado a la muestra		

DIAGRAMAS ESTRUCTURADOS

Diagrama 1. Proceso general del sistema

	PARCIAL 3ER CORTE DIAGRAMA ESTRUCTURADO GENERAL DEL SISTEMA
1	**INICIO**
2	IMPORTAR librerías OS, IPyWidgets e IPython.display
3	DEFINIR archivo de almacenamiento 'inspecciones_ifn.txt'
4	DEFINIR funciones de la capa de datos
5	DEFINIR funciones de la capa de lógica de negocio
6	CREAR campos, botones y espacios de salida
7	CREAR pestaña REGISTRAR
8	CREAR pestaña CONSULTAR
9	CREAR pestaña INFORMES
10	ORGANIZAR las tres pestañas en la interfaz principal
11	MOSTRAR encabezado y ventana principal
12	**MIENTRAS el usuario interactúe con el sistema HACER**
13	LEER pestaña seleccionada por el usuario
14	**SI pestaña seleccionada == REGISTRAR ENTONCES**
15	LEER ID, geolocalización y nombreBrigada
16	LEER fustales, latizales y brinzales
17	LEER nombreComunEspecie, usoEspecie y codigoMuestra
18	**SI algún campo obligatorio está vacío ENTONCES**
19	MOSTRAR 'Complete todos los campos antes de registrar.'
20	**SINO**
21	ORGANIZAR los nueve datos en una cadena de texto
22	ABRIR archivo 'inspecciones_ifn.txt' en modo adición
23	ESCRIBIR registro separado por '|'
24	CERRAR archivo
25	**SI almacenamiento exitoso ENTONCES**
26	MOSTRAR 'Inspección guardada correctamente.'
27	LIMPIAR campos del formulario
28	**SINO**
29	MOSTRAR 'Error al guardar la inspección.'
30	**FIN SI**
31	**FIN SI**
32	**SINO SI pestaña seleccionada == CONSULTAR ENTONCES**
33	LEER ID del conglomerado a consultar
34	**SI ID está vacío ENTONCES**
35	MOSTRAR 'Ingrese el ID del conglomerado.'
36	**SINO**
37	LEER registros del archivo TXT
38	ENCONTRADO = FALSO
39	**PARA CADA línea del archivo HACER**
40	CONVERTIR línea en diccionario de inspección
41	**SI registro es válido ENTONCES**
42	**SI idConglomerado == ID buscado ENTONCES**
43	ENCONTRADO = VERDADERO
44	MOSTRAR información de la inspección
45	FINALIZAR búsqueda
46	**FIN SI**
47	**FIN SI**
48	**FIN PARA**
49	**SI ENCONTRADO == FALSO ENTONCES**
50	MOSTRAR 'No se encontró una inspección con ese ID.'
51	**FIN SI**
52	**FIN SI**
53	**SINO SI pestaña seleccionada == INFORMES ENTONCES**
54	OBTENER todas las inspecciones válidas
55	**SI no existen inspecciones ENTONCES**
56	MOSTRAR 'No existen inspecciones registradas.'
57	**SINO**
58	TOTAL_INSPECCIONES = cantidad de inspecciones
59	TOTAL_ARBOLES = 0
60	CREAR conjunto de especies
61	CREAR conjunto de usos
62	**PARA CADA inspección HACER**
63	SUMAR fustales + latizales + brinzales
64	AGREGAR especie al conjunto de especies
65	AGREGAR uso al conjunto de usos
66	**FIN PARA**
67	CALCULAR densidad = TOTAL_ARBOLES / TOTAL_INSPECCIONES
68	ORDENAR especies y usos alfabéticamente
69	MOSTRAR total de inspecciones
70	MOSTRAR total de árboles
71	MOSTRAR densidad promedio
72	MOSTRAR especies presentes y usos registrados
73	**FIN SI**
74	**FIN SI**
75	**FIN MIENTRAS**
76	FINALIZAR interacción con el sistema
77	**FIN**

Diagrama 2. Proceso de registro de inspecciones

	PARCIAL 3ER CORTE DIAGRAMA ESTRUCTURADO DEL REGISTRO DE INSPECCIONES
1	**INICIO**
2	MOSTRAR formulario de registro de inspecciones
3	ESPERAR clic en el botón 'Registrar inspección'
4	LIMPIAR espacio de salida
5	LEER ID del conglomerado
6	LEER geolocalización
7	LEER nombre de la brigada
8	LEER cantidad de fustales, latizales y brinzales
9	LEER nombre común de la especie
10	LEER uso de la especie
11	LEER código de muestra
12	**SI algún campo obligatorio está vacío ENTONCES**
13	MOSTRAR 'Complete todos los campos antes de registrar.'
14	**SINO**
15	ORGANIZAR los nueve datos en una cadena de texto
16	SEPARAR cada dato mediante el carácter '|'
17	ABRIR archivo 'inspecciones_ifn.txt' en modo adición
18	ESCRIBIR registro en el archivo TXT
19	CERRAR archivo
20	**SI registro guardado correctamente ENTONCES**
21	MOSTRAR '¡Éxito! La inspección fue guardada correctamente.'
22	LIMPIAR ID del conglomerado
23	LIMPIAR geolocalización
24	LIMPIAR nombre de la brigada
25	RESTABLECER fustales, latizales y brinzales a 0
26	LIMPIAR nombre común de la especie
27	RESTABLECER uso de la especie
28	LIMPIAR código de muestra
29	**SINO**
30	MOSTRAR 'Error: no fue posible guardar la inspección.'
31	**FIN SI**
32	**FIN SI**
33	**FIN**

Diagrama 3. Proceso de consulta de inspecciones

	PARCIAL 3ER CORTE DIAGRAMA ESTRUCTURADO DE CONSULTA DE INSPECCIONES
1	**INICIO**
2	MOSTRAR formulario de consulta
3	ESPERAR clic en el botón 'Consultar inspección'
4	LIMPIAR espacio de salida
5	LEER ID del conglomerado a consultar
6	**SI ID está vacío ENTONCES**
7	MOSTRAR 'Ingrese el ID del conglomerado.'
8	**SINO**
9	ABRIR archivo 'inspecciones_ifn.txt' en modo lectura
10	LEER todas las líneas del archivo
11	CERRAR archivo
12	ENCONTRADO = FALSO
13	**PARA CADA línea del archivo HACER**
14	ELIMINAR espacios y saltos de línea
15	SEPARAR los campos mediante '|'
16	**SI cantidad de campos == 9 ENTONCES**
17	CONVERTIR línea en diccionario
18	CONVERTIR fustales, latizales y brinzales a enteros
19	**SI conversión de datos es válida ENTONCES**
20	**SI idConglomerado == ID buscado ENTONCES**
21	ENCONTRADO = VERDADERO
22	OBTENER información de la inspección
23	MOSTRAR datos en una tabla
24	FINALIZAR búsqueda
25	**FIN SI**
26	**FIN SI**
27	**FIN SI**
28	**FIN PARA**
29	**SI ENCONTRADO == FALSO ENTONCES**
30	MOSTRAR 'No se encontró una inspección con ese ID.'
31	**FIN SI**
32	**FIN SI**
33	**FIN**

Diagrama 4. Proceso de generación de informes

	PARCIAL 3ER CORTE DIAGRAMA ESTRUCTURADO DE GENERACIÓN DE INFORMES E INDICADORES
1	**INICIO**
2	MOSTRAR pestaña de informes e indicadores
3	ESPERAR clic en el botón 'Generar informe'
4	LIMPIAR espacio de salida
5	LEER todas las inspecciones del archivo TXT
6	CONVERTIR cada línea válida en un diccionario
7	DESCARTAR registros inválidos
8	**SI no existen inspecciones válidas ENTONCES**
9	MOSTRAR 'No existen inspecciones registradas para calcular indicadores.'
10	**SINO**
11	TOTAL_INSPECCIONES = cantidad de inspecciones
12	TOTAL_ARBOLES = 0
13	CREAR conjunto ESPECIES_PRESENTES
14	CREAR conjunto USOS_PRESENTES
15	**PARA CADA inspección HACER**
16	OBTENER cantidad de fustales
17	OBTENER cantidad de latizales
18	OBTENER cantidad de brinzales
19	TOTAL_ARBOLES = TOTAL_ARBOLES + fustales + latizales + brinzales
20	AGREGAR nombreComunEspecie a ESPECIES_PRESENTES
21	AGREGAR usoEspecie a USOS_PRESENTES
22	**FIN PARA**
23	CALCULAR DENSIDAD = TOTAL_ARBOLES / TOTAL_INSPECCIONES
24	ORDENAR ESPECIES_PRESENTES alfabéticamente
25	ORDENAR USOS_PRESENTES alfabéticamente
26	MOSTRAR 'Informe regional e indicadores forestales (IFN)'
27	MOSTRAR total de conglomerados evaluados
28	MOSTRAR total de árboles contados
29	MOSTRAR densidad promedio de árboles por conglomerado
30	MOSTRAR especies presentes
31	MOSTRAR usos registrados
32	**FIN SI**
33	**FIN**

DISEÑO DE ALGORITMO (1) TABLA DE IDENTIFICADORES

Aquí tienes la tabla detallada con las **variables**, **constantes**, **tipos de dato** y su **uso (entrada, procesamiento o salida)** dentro del programa del Inventario Forestal Nacional (IFN):

Nombre de Elemento	**Tipo (Constante / Variable)**	**Tipo de Dato**	**Uso / Descripción en el Programa**
NOMBRE_ARCHIVO	Constante	str (Cadena de texto)	Almacena el nombre fijo del archivo plano ("inspecciones_ifn.txt") utilizado para la persistencia.
idConglomerado	Variable	str (Cadena de texto)	**Entrada:** Identificación única asignada al conglomerado forestal evaluado.
geolocalizacion	Variable	str (Cadena de texto)	**Entrada:** Verificación de la ubicación y subparcelas desde el centroide del conglomerado.
nombreBrigada	Variable	str (Cadena de texto)	**Entrada:** Nombre de la brigada de levantamiento de información asignada en campo.
fustales	Variable	int (Entero)	**Entrada / Procesamiento:** Cantidad de árboles contados clasificados como fustales según su diámetro.
latizales	Variable	int (Entero)	**Entrada / Procesamiento:** Cantidad de árboles contados clasificados como latizales según su diámetro.
brinzales	Variable	int (Entero)	**Entrada / Procesamiento:** Cantidad de árboles contados clasificados como brinzales según su diámetro.

nombreComunEspecie	Variable	str (Cadena de texto)	**Entrada:** Nombre común de la especie registrada durante la toma de muestras.
usoEspecie	Variable	str (Cadena de texto)	**Entrada:** Clasificación del uso de la especie (*medicinal, alimenticio o maderable*).
codigoMuestra	Variable	str (Cadena de texto)	**Entrada:** Código de identificación asignado a la muestra botánica.
opcionMenu	Variable	str (Cadena de texto)	**Entrada de control:** Almacena la opción numérica seleccionada por el usuario en la consola (1 al 4).
idBuscado	Variable	str (Cadena de texto)	**Entrada:** ID del conglomerado que el usuario desea buscar y consultar en el sistema.
registro_linea	Variable	str (Cadena de texto)	**Procesamiento:** Trama de texto unida por delimitadores (`
lineas	Variable	list (Lista de cadenas)	**Procesamiento:** Contiene todas las líneas leídas desde el archivo de texto plano para ser analizadas.
inspeccion / inspeccion_encontrada	Variable	dict (Diccionario)	**Procesamiento / Salida:** Estructura que empaqueta y mapea los campos de una inspección particular tras ser filtrada por ID.
lista_inspecciones	Variable	list (Lista de diccionarios)	**Procesamiento:** Almacena el conjunto de todas las inspecciones recuperadas del archivo plano para cálculos globales.
total_inspecciones	Variable	int (Entero)	**Procesamiento / Salida:** Conteo total

			de conglomerados evaluados en los informes.
total_arboles	Variable	int (Entero)	**Procesamiento / Salida:** Sumatoria acumulada de todos los árboles (*fustales* + *latizales* + *brinzales*) contados.
densidad_cobertura_vegetal	Variable	float (Decimal)	**Procesamiento / Salida:** Indicador calculado como el promedio de árboles por conglomerado.
especies_presentes	Variable	set (Conjunto)	**Procesamiento / Salida:** Colección que agrupa sin repetir los nombres comunes de las especies encontradas.
usos_presentes	Variable	set (Conjunto)	**Procesamiento / Salida:** Colección que agrupa sin repetir los usos registrados en las inspecciones.

DISEÑO DE ALGORITMO (2) DIAGRAMAS DE FLUJO

Diagrama 1. Proceso general del sistema

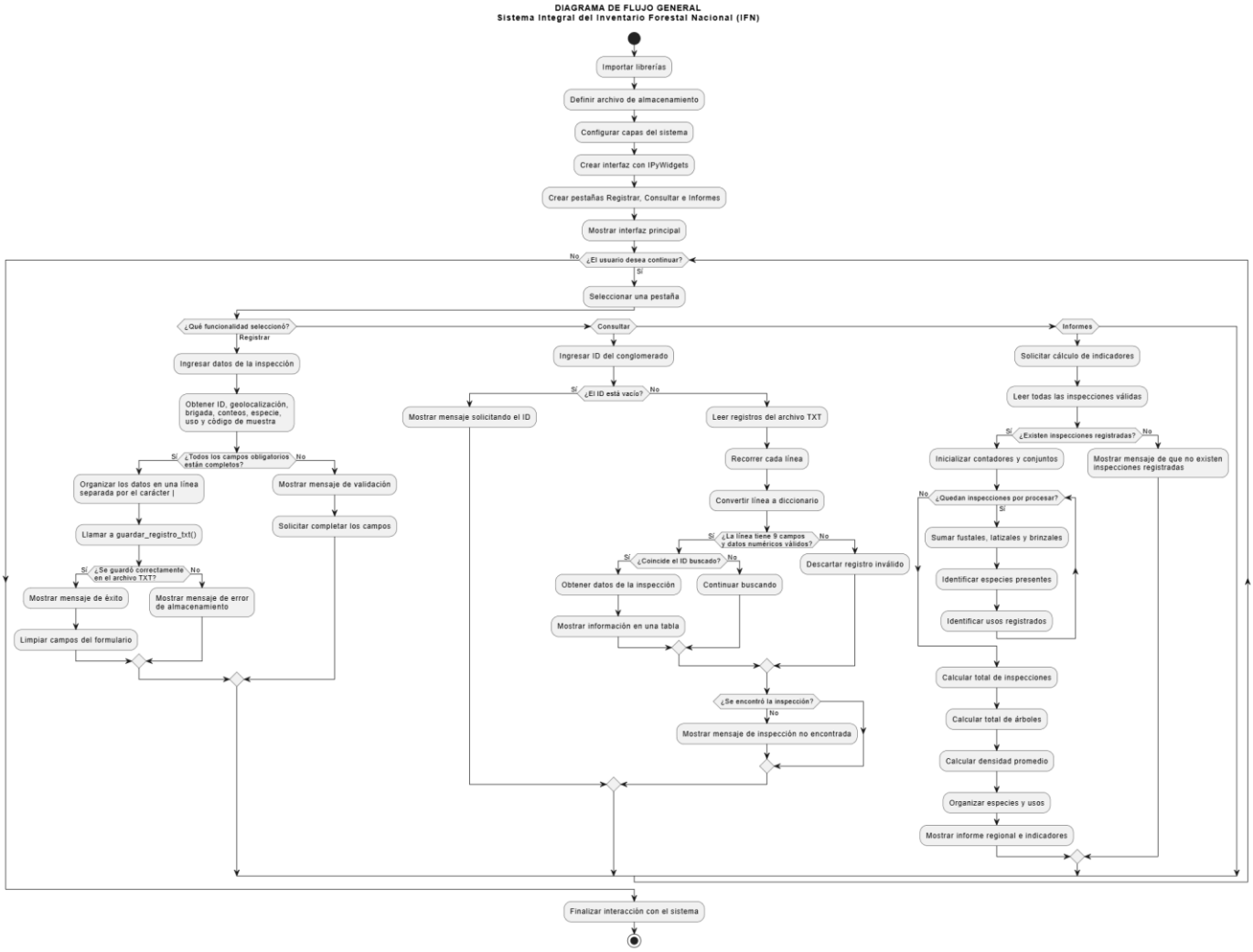


Diagrama 2. Proceso de registro de inspecciones

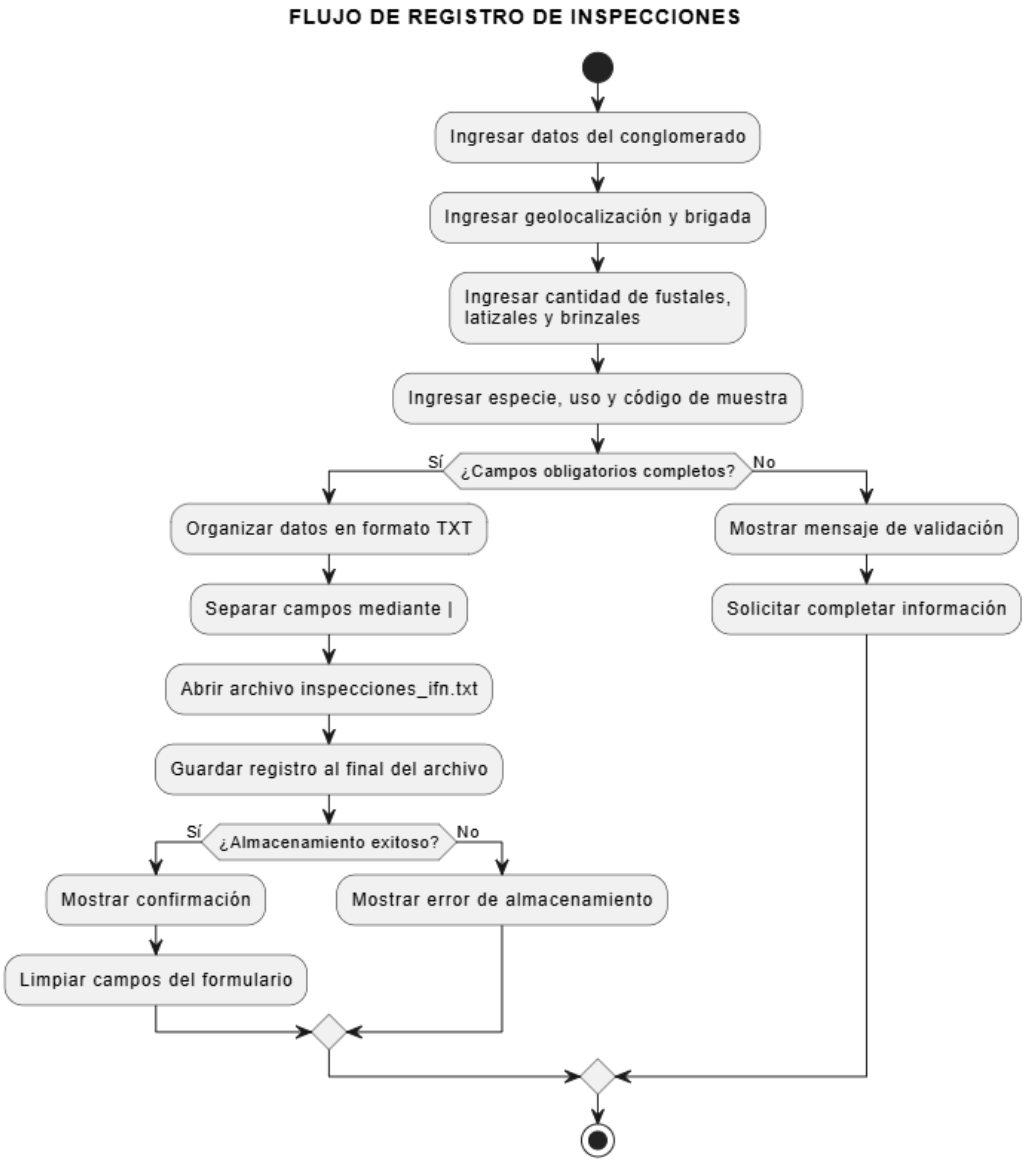


Diagrama 3. Proceso de consulta de inspecciones

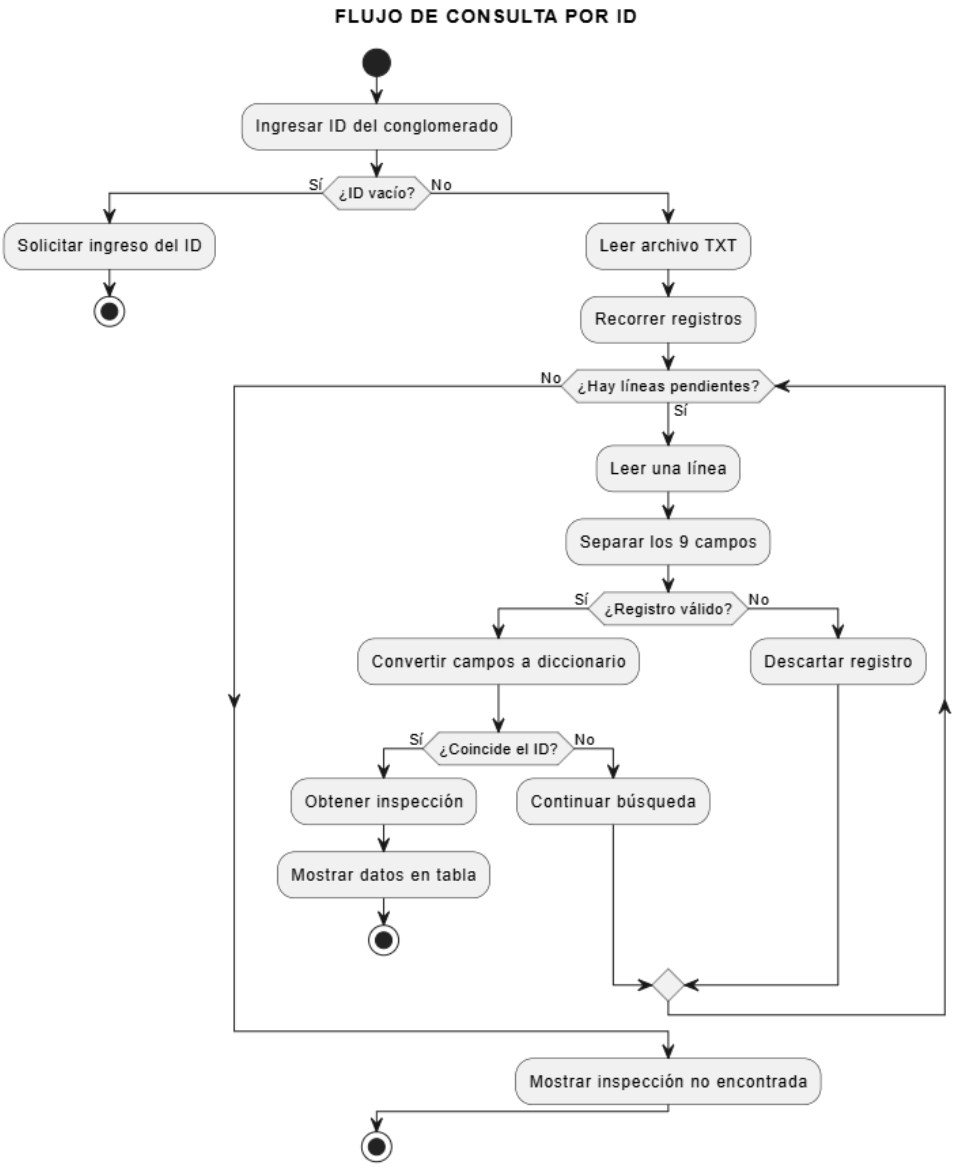
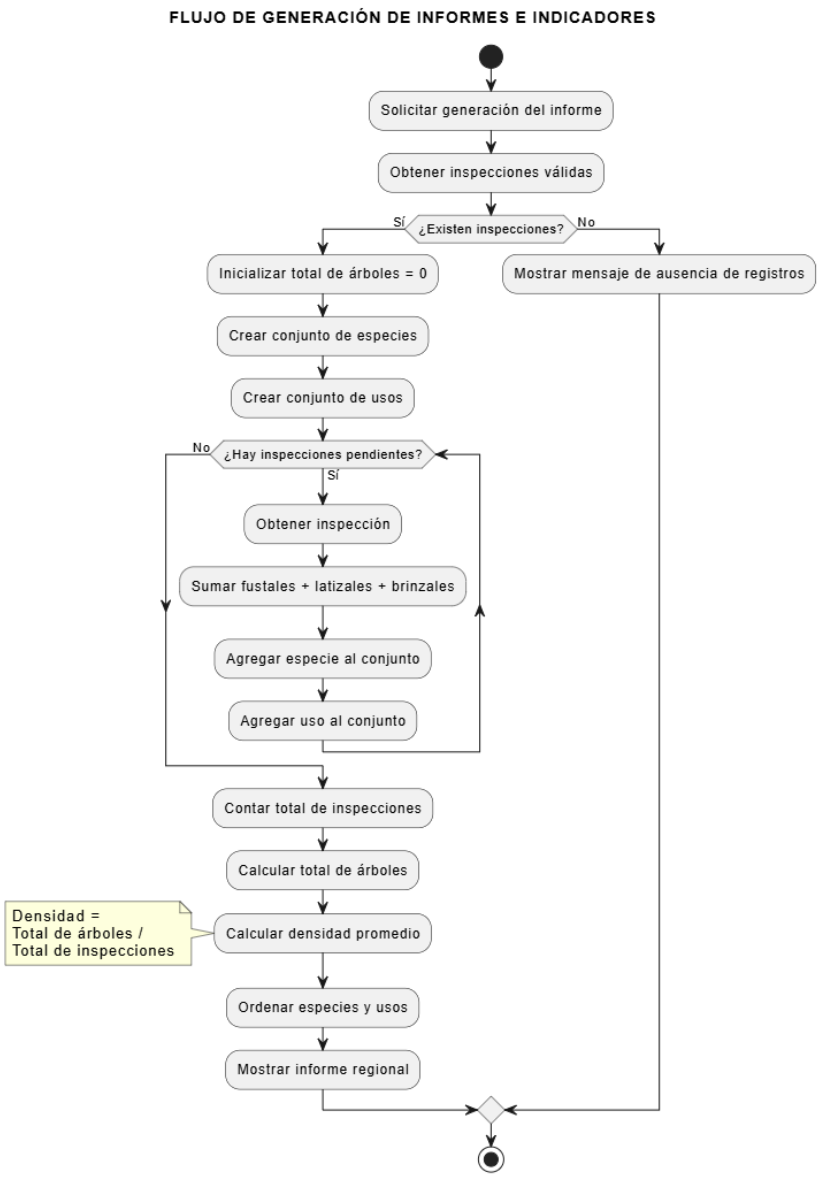


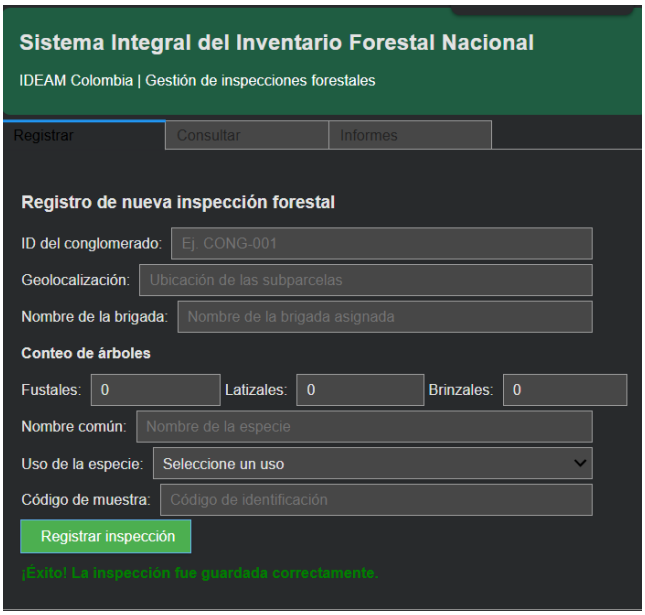
Diagrama 4. Proceso de generación de informes



PRUEBA DE DISEÑO

1. Registro de inspecciones

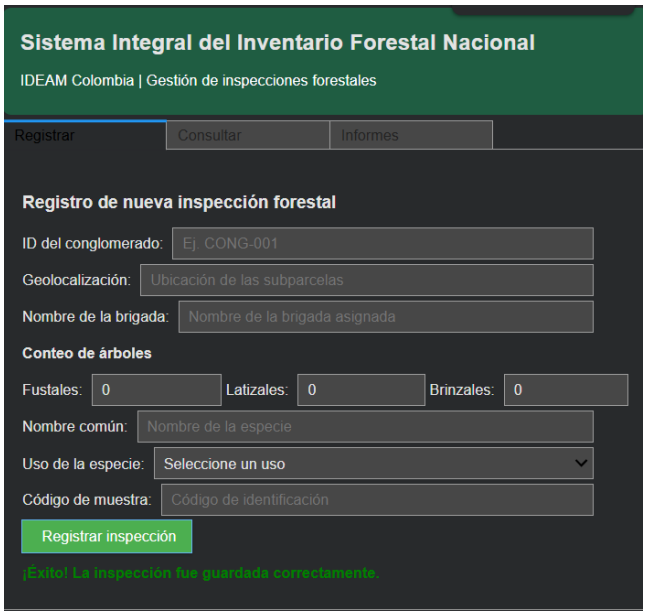
1. Figura 1. Registro exitoso de una nueva inspección forestal. (CP-01)



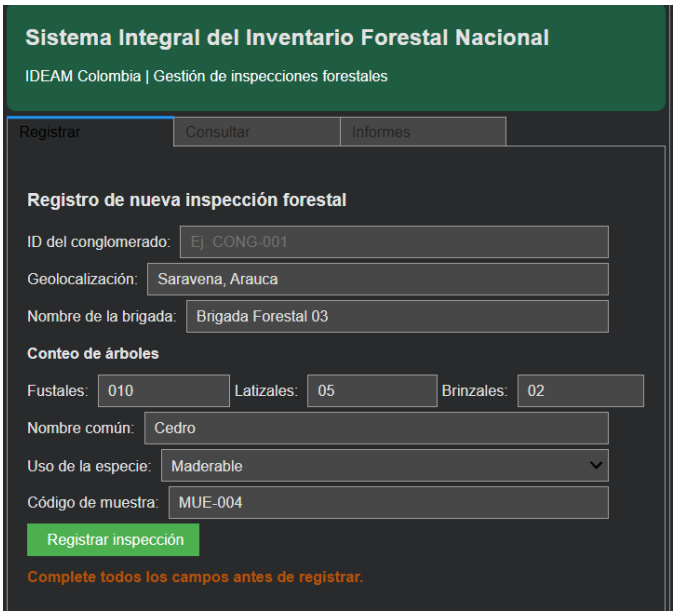
2. Figura 2. Registro de inspección con cantidades de árboles iguales a cero. (CP-02)



3. Figura 3. Registro de inspección con cantidades elevadas de individuos forestales. (CP-03)



4. Figura 4. Validación de campos obligatorios durante el registro de inspecciones. (CP-04)



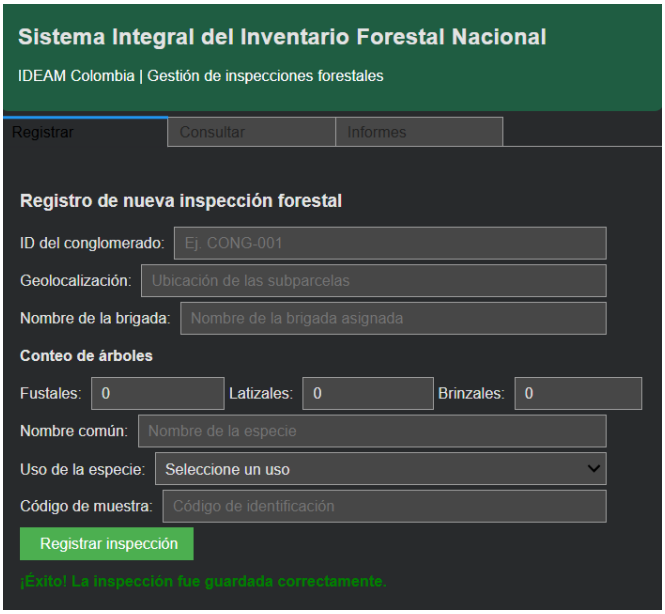
5. Figura 5. Validación de geolocalización obligatoria. (CP-05)



6. Figura 6. Validación de selección del uso de la especie forestal. (CP-06)



7. Figura 7. Validación de valores negativos en el conteo de árboles. (CP-07)



Nota: Corrige automáticamente el número negativo en 0 (antes de “Registrar inspección)

8. Figura 8. Registro de inspecciones con diferentes especies forestales. (CP-08)



2. Consulta de inspecciones

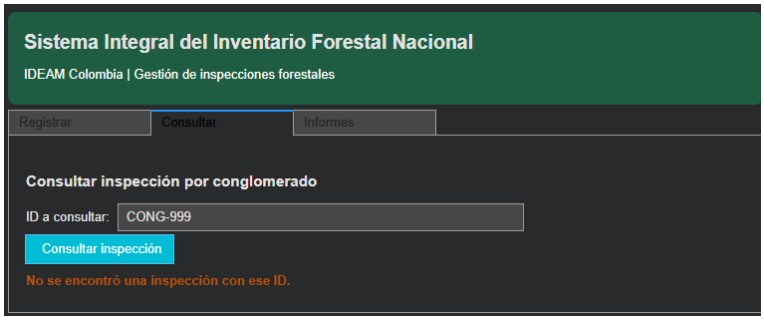
1. Figura 9. Consulta exitosa de una inspección mediante el ID del conglomerado. (CP-09)



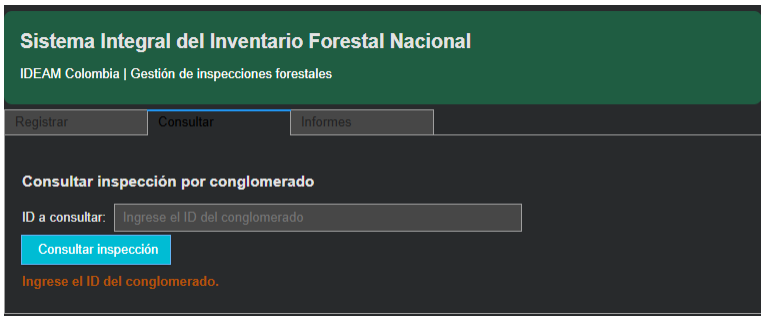
2. Figura 10. Consulta de una segunda inspección registrada en el sistema. (CP-10)



3. Figura 11. Validación de consulta de un conglomerado inexistente. (CP-11)



4. Figura 12. Validación de consulta con el campo de identificación vacío. (CP-12)

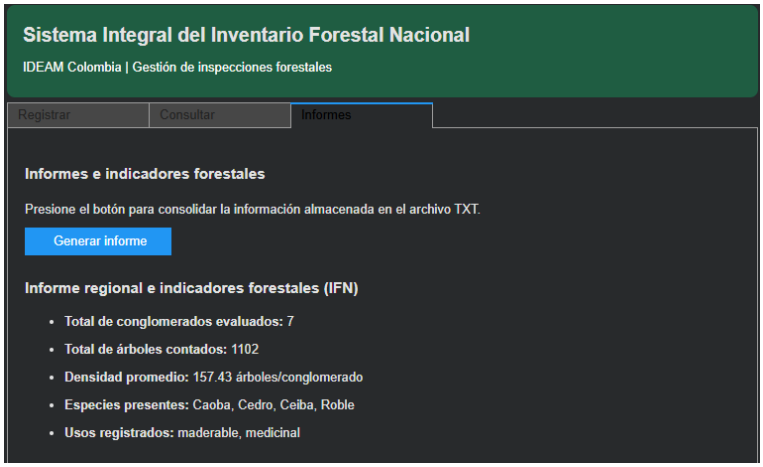


5. Figura 13. Consulta de inspección mediante un ID con espacios adicionales. (CP-13)



3. Generación de informes e indicadores

1. Figura 14. Generación del informe consolidado de inspecciones forestales. (CP-14)

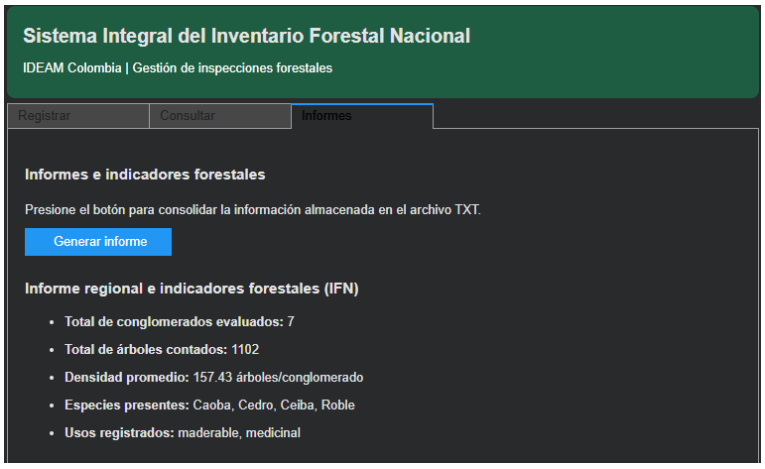


2. Figura 15. Verificación del cálculo de la densidad promedio de cobertura vegetal. (CP-15)

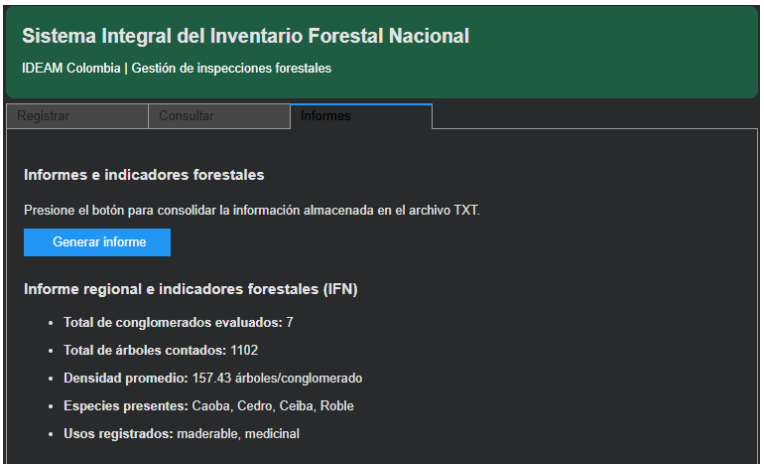
   Total árboles: 1102

   Total inspecciones: 7

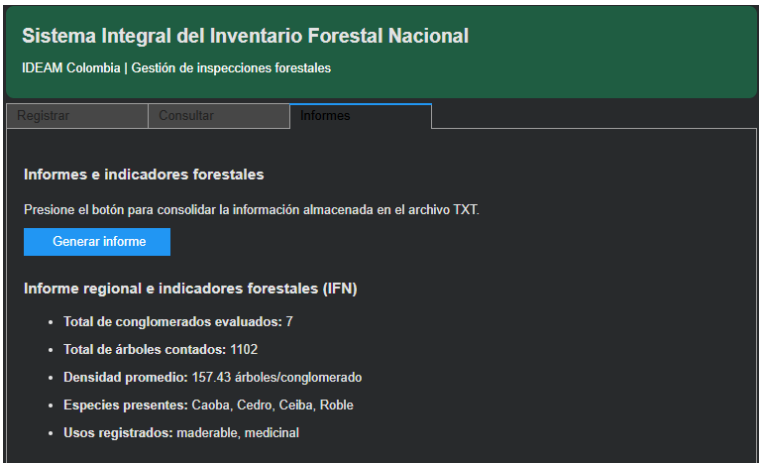
$$D = \frac{1102}{7} = 157{,}43$$



3. Figura 16. Visualización de las especies forestales presentes en los registros. (CP-16)



4. Figura 17. Visualización de los usos registrados para las especies forestales. (CP-17)

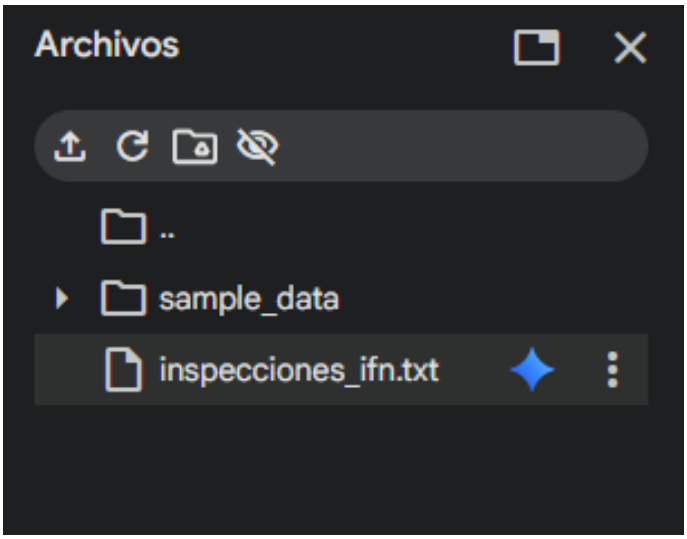


5. Figura 18. Validación de generación de informes sin inspecciones registradas. (CP-18)

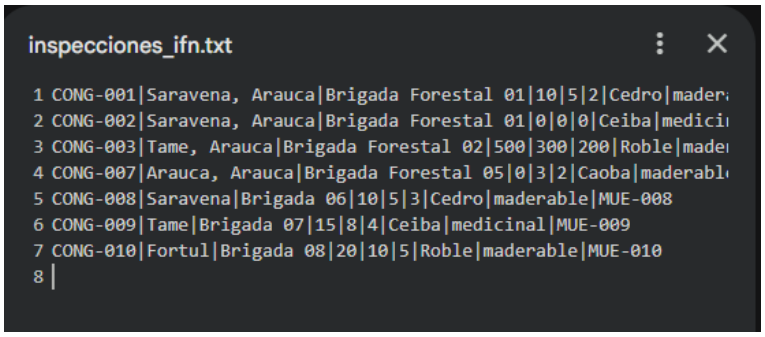


4. Almacenamiento y persistencia de datos

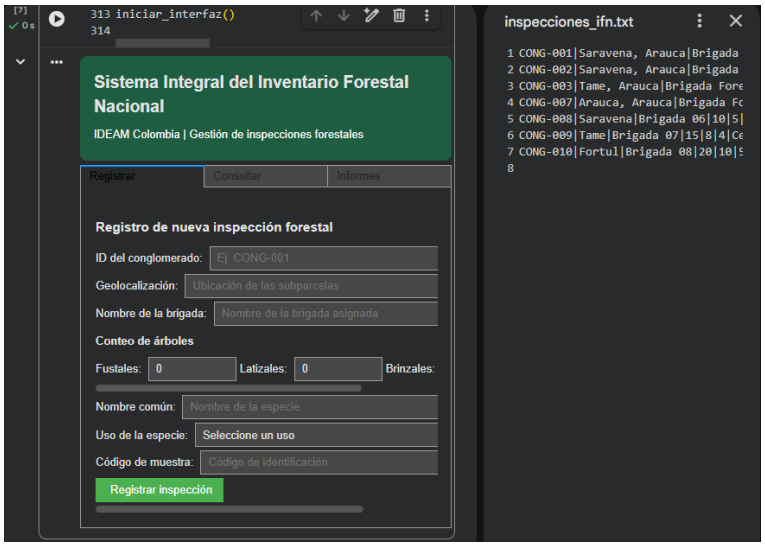
1. Figura 19. Verificación de la creación del archivo de almacenamiento TXT. (CP-19)



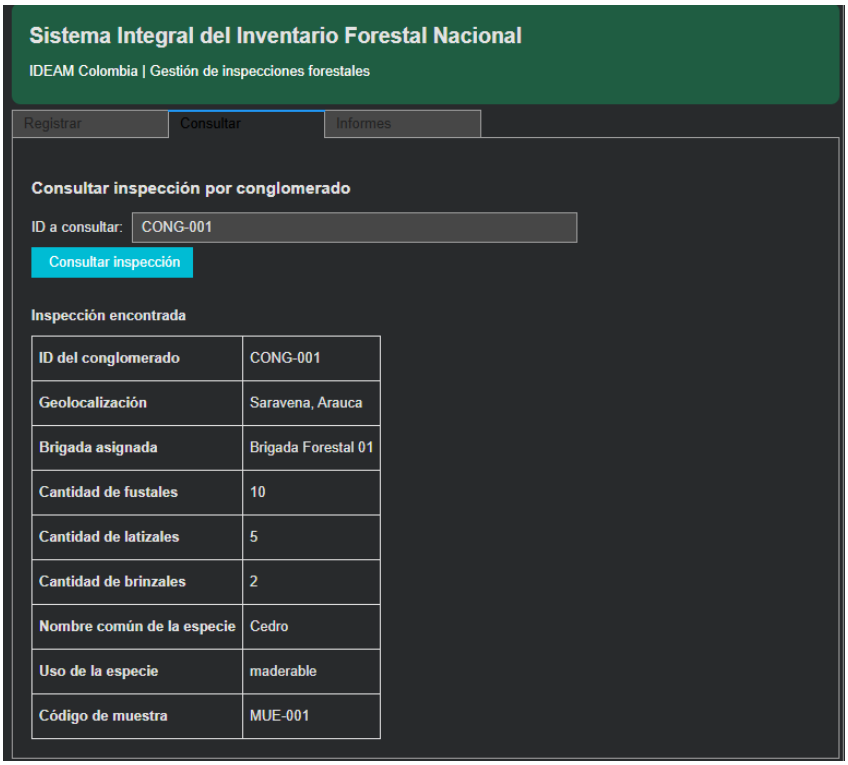
2. Figura 20. Verificación de la estructura y almacenamiento de los registros en el archivo TXT. (CP-20)



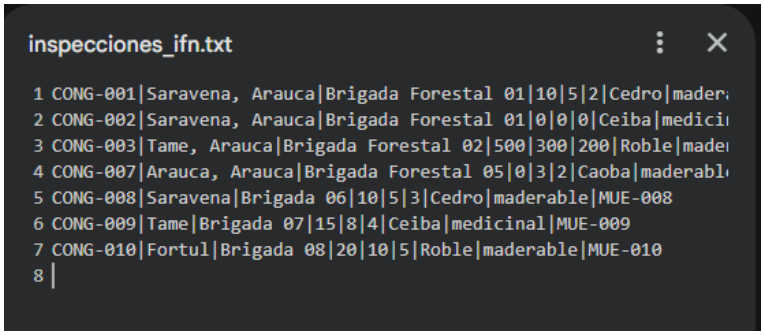
3. Figura 21. Verificación de persistencia de los datos después de reiniciar el programa. (CP-21)



4. Figura 22. Verificación del almacenamiento de múltiples inspecciones forestales. (CP-22)

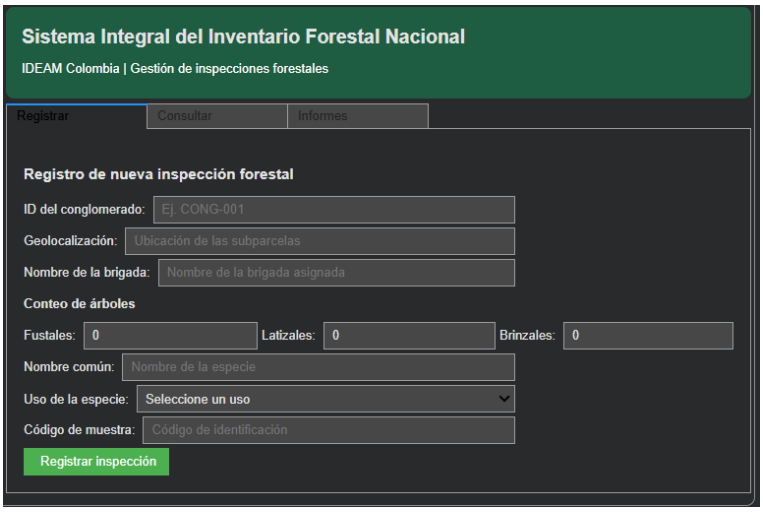


5. Figura 23. Verificación de recuperación y lectura de registros almacenados. (CP-23)

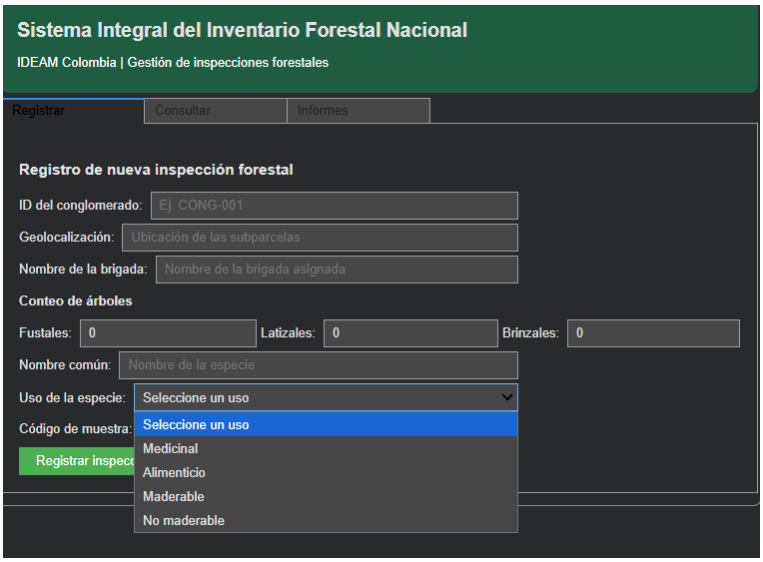


5. Interfaz gráfica IPyWidgets

1. Figura 24. Visualización de las pestañas de navegación de la interfaz IPyWidgets. (CP-24)



2. Figura 25. Funcionamiento del menú desplegable para la selección del uso de las especies. (CP-26)



CÓDIGO

```
#
================================================================================
# PROYECTO: INVENTARIO FORESTAL NACIONAL (IFN) - IDEAM
# MATERIA: Programación de Computadores I
# INTERFAZ: IPyWidgets para Jupyter Notebook / JupyterLab
# ESTUDIANTES: DAVID SANTIAGO GUERRERO GARCIA - NIKOL FERNANDA MEDINA MONTERREY
#
================================================================================

import os
import ipywidgets as widgets
from IPython.display import display, clear_output

# Archivo de persistencia (Capa de Datos)
NOMBRE_ARCHIVO = "inspecciones_ifn.txt"


#
================================================================================
# 1. CAPA DE DATOS
#
================================================================================

def guardar_registro_txt(registro_linea):
    """Guarda una inspección al final del archivo de texto."""
    try:
        with open(NOMBRE_ARCHIVO, "a", encoding="utf-8") as archivo:
            archivo.write(registro_linea + "\n")
        return True
    except IOError as error:
        print(f"Error al escribir en el archivo: {error}")
        return False


def leer_registros_txt():
    """Lee todas las líneas del archivo; si no existe, retorna una lista vacía."""
    if not os.path.exists(NOMBRE_ARCHIVO):
```

```
        return []
    try:
        with open(NOMBRE_ARCHIVO, "r", encoding="utf-8") as archivo:
            return archivo.readlines()
    except IOError as error:
        print(f"Error al leer el archivo: {error}")
        return []


#
================================================================================
======
# 2. CAPA DE LÓGICA DE NEGOCIO
#
================================================================================
======

def convertir_linea_a_inspeccion(linea):
    """Convierte una línea del TXT en un diccionario de inspección."""
    campos = linea.strip().split("|")
    if len(campos) != 9:
        return None
    try:
        return {
            "idConglomerado": campos[0],
            "geolocalizacion": campos[1],
            "nombreBrigada": campos[2],
            "fustales": int(campos[3]),
            "latizales": int(campos[4]),
            "brinzales": int(campos[5]),
            "nombreComunEspecie": campos[6],
            "usoEspecie": campos[7],
            "codigoMuestra": campos[8]
        }
    except ValueError:
        return None


def procesar_nueva_inspeccion(idConglomerado, geolocalizacion,
nombreBrigada,
                              fustales, latizales, brinzales,
                              nombreComunEspecie, usoEspecie,
codigoMuestra):
    """Organiza los datos y solicita su almacenamiento a la capa de
datos."""
```

```
    registro_linea = (
        f"{idConglomerado}|{geolocalizacion}|{nombreBrigada}|"
        f"{fustales}|{latizales}|{brinzales}|"
        f"{nombreComunEspecie}|{usoEspecie}|{codigoMuestra}"
    )
    return guardar_registro_txt(registro_linea)


def buscar_inspeccion_logica(idBuscado):
    """Busca una inspección por el ID del conglomerado."""
    for linea in leer_registros_txt():
        inspeccion = convertir_linea_a_inspeccion(linea)
        if inspeccion and inspeccion["idConglomerado"] == idBuscado:
            return inspeccion
    return None


def obtener_todas_las_inspecciones_logica():
    """Retorna todas las inspecciones válidas almacenadas en el
TXT."""
    lista_inspecciones = []
    for linea in leer_registros_txt():
        inspeccion = convertir_linea_a_inspeccion(linea)
        if inspeccion:
            lista_inspecciones.append(inspeccion)
    return lista_inspecciones


def calcular_indicadores():
    """Calcula los totales e indicadores generales del inventario."""
    inspecciones = obtener_todas_las_inspecciones_logica()
    if not inspecciones:
        return None

    total_inspecciones = len(inspecciones)
    total_arboles = 0
    especies_presentes = set()
    usos_presentes = set()

    for inspeccion in inspecciones:
        total_arboles += (
            inspeccion["fustales"] +
            inspeccion["latizales"] +
            inspeccion["brinzales"]
        )
```

```
        especies_presentes.add(inspeccion["nombreComunEspecie"])
        usos_presentes.add(inspeccion["usoEspecie"])

    densidad = total_arboles / total_inspecciones
    return {
        "total_inspecciones": total_inspecciones,
        "total_arboles": total_arboles,
        "densidad": densidad,
        "especies": sorted(especies_presentes),
        "usos": sorted(usos_presentes)
    }


#
================================================================================
======
# 3. CAPA DE PRESENTACIÓN: INTERFAZ CON IPYWIDGETS
#
================================================================================
======

# Estilo general
estilo = {"description_width": "initial"}

# ----- Pestaña 1: Registro de inspecciones -----
id_widget = widgets.Text(description="ID del conglomerado:",
placeholder="Ej. CONG-001", style=estilo,
layout=widgets.Layout(width="500px"))
geo_widget = widgets.Text(description="Geolocalización:",
placeholder="Ubicación de las subparcelas", style=estilo,
layout=widgets.Layout(width="500px"))
brigada_widget = widgets.Text(description="Nombre de la brigada:",
placeholder="Nombre de la brigada asignada", style=estilo,
layout=widgets.Layout(width="500px"))
fustales_widget = widgets.BoundedIntText(
    description="Fustales:",
    min=0,
    max=1000000,
    value=0,
    style=estilo,
    layout=widgets.Layout(width="250px")
)

latizales_widget = widgets.BoundedIntText(
    description="Latizales:",
```

```
    min=0,
    max=1000000,
    value=0,
    style=estilo,
    layout=widgets.Layout(width="250px")
)

brinzales_widget = widgets.BoundedIntText(
    description="Brinzales:",
    min=0,
    max=1000000,
    value=0,
    style=estilo,
    layout=widgets.Layout(width="250px")
)
especie_widget = widgets.Text(description="Nombre común:",
placeholder="Nombre de la especie", style=estilo,
layout=widgets.Layout(width="500px"))
uso_widget = widgets.Dropdown(
    options=[("Seleccione un uso", ""), ("Medicinal", "medicinal"),
             ("Alimenticio", "alimenticio"), ("Maderable",
"maderable"),
             ("No maderable", "no maderable")],
    description="Uso de la especie:", style=estilo,
layout=widgets.Layout(width="500px")
)
codigo_widget = widgets.Text(description="Código de muestra:",
placeholder="Código de identificación", style=estilo,
layout=widgets.Layout(width="500px"))
registrar_boton = widgets.Button(description="Registrar inspección",
button_style="success", icon="save")
registro_salida = widgets.Output()


def registrar_inspeccion(_):
    with registro_salida:
        clear_output()
        datos_texto = [id_widget.value, geo_widget.value,
brigada_widget.value,
                       especie_widget.value, codigo_widget.value]
        if any(not dato.strip() for dato in datos_texto) or not
uso_widget.value:
            display(widgets.HTML("<span
style='color:#b45309'><b>Complete todos los campos antes de
registrar.</b></span>"))
```

```
            return

        resultado = procesar_nueva_inspeccion(
            id_widget.value.strip(), geo_widget.value.strip(),
brigada_widget.value.strip(),
            fustales_widget.value, latizales_widget.value,
brinzales_widget.value,
            especie_widget.value.strip(), uso_widget.value,
codigo_widget.value.strip()
        )
        if resultado:
            display(widgets.HTML("<span style='color:green'><b>¡Éxito!
La inspección fue guardada correctamente.</b></span>"))
            # Limpiar los campos después de guardar
            for campo in [id_widget, geo_widget, brigada_widget,
especie_widget, codigo_widget]:
                campo.value = ""
            fustales_widget.value = 0
            latizales_widget.value = 0
            brinzales_widget.value = 0
            uso_widget.value = ""
        else:
            display(widgets.HTML("<span style='color:red'><b>Error: no
fue posible guardar la inspección.</b></span>"))

registrar_boton.on_click(registrar_inspeccion)

formulario_registro = widgets.VBox([
    widgets.HTML("<h3>Registro de nueva inspección forestal</h3>"),
    id_widget, geo_widget, brigada_widget,
    widgets.HTML("<b>Conteo de árboles</b>"),
    widgets.HBox([fustales_widget, latizales_widget,
brinzales_widget]),
    especie_widget, uso_widget, codigo_widget,
    registrar_boton, registro_salida
], layout=widgets.Layout(gap="8px"))


# ----- Pestaña 2: Consulta por ID -----
consulta_id_widget = widgets.Text(description="ID a consultar:",
placeholder="Ingrese el ID del conglomerado", style=estilo,
layout=widgets.Layout(width="500px"))
consultar_boton = widgets.Button(description="Consultar inspección",
button_style="info", icon="search")
consulta_salida = widgets.Output()
```

```
def consultar_inspeccion(_):
    with consulta_salida:
        clear_output()
        id_buscado = consulta_id_widget.value.strip()
        if not id_buscado:
            display(widgets.HTML("<span
style='color:#b45309'><b>Ingrese el ID del conglomerado.</b></span>"))
            return

        inspeccion = buscar_inspeccion_logica(id_buscado)
        if inspeccion:
            etiquetas = {
                "idConglomerado": "ID del conglomerado",
                "geolocalizacion": "Geolocalización",
                "nombreBrigada": "Brigada asignada",
                "fustales": "Cantidad de fustales",
                "latizales": "Cantidad de latizales",
                "brinzales": "Cantidad de brinzales",
                "nombreComunEspecie": "Nombre común de la especie",
                "usoEspecie": "Uso de la especie",
                "codigoMuestra": "Código de muestra"
            }
            filas = "".join(
                f"<tr><th style='text-
align:left;padding:6px'>{etiquetas[clave]}</th>"
                f"<td style='padding:6px'>{valor}</td></tr>"
                for clave, valor in inspeccion.items()
            )
            display(widgets.HTML(
                "<h4>Inspección encontrada</h4>"
                f"<table style='border-collapse:collapse'
border='1'>{filas}</table>"
            ))
        else:
            display(widgets.HTML("<span style='color:#b45309'><b>No se
encontró una inspección con ese ID.</b></span>"))

consultar_boton.on_click(consultar_inspeccion)
formulario_consulta = widgets.VBox([
    widgets.HTML("<h3>Consultar inspección por conglomerado</h3>"),
    consulta_id_widget, consultar_boton, consulta_salida
], layout=widgets.Layout(gap="10px"))
```

```
# ----- Pestaña 3: Informes e indicadores -----
informe_boton = widgets.Button(description="Generar informe",
button_style="primary", icon="bar-chart")
informe_salida = widgets.Output()


def generar_informe(_):
    with informe_salida:
        clear_output()
        indicadores = calcular_indicadores()
        if indicadores is None:
            display(widgets.HTML("<b>No existen inspecciones
registradas para calcular indicadores.</b>"))
            return

        display(widgets.HTML(f"""
        <h3>Informe regional e indicadores forestales (IFN)</h3>
        <ul>
          <li><b>Total de conglomerados evaluados:</b>
{indicadores['total_inspecciones']}</li>
          <li><b>Total de árboles contados:</b>
{indicadores['total_arboles']}</li>
          <li><b>Densidad promedio:</b> {indicadores['densidad']:.2f}
árboles/conglomerado</li>
          <li><b>Especies presentes:</b> {',
'.join(indicadores['especies'])}</li>
          <li><b>Usos registrados:</b> {',
'.join(indicadores['usos'])}</li>
        </ul>
        """))

informe_boton.on_click(generar_informe)
formulario_informe = widgets.VBox([
    widgets.HTML("<h3>Informes e indicadores forestales</h3>"),
    widgets.HTML("Presione el botón para consolidar la información
almacenada en el archivo TXT."),
    informe_boton, informe_salida
], layout=widgets.Layout(gap="10px"))


# ----- Ventana principal con pestañas -----
encabezado = widgets.HTML("""
<div style='background:#1f5d42;color:white;padding:16px;border-
radius:8px'>
```

```
  <h2 style='margin:0'>Sistema Integral del Inventario Forestal
Nacional</h2>
  <p style='margin:5px 0 0 0'>IDEAM Colombia | Gestión de inspecciones
forestales</p>
</div>
""")
pestanas = widgets.Tab(children=[formulario_registro,
formulario_consulta, formulario_informe])
pestanas.set_title(0, "Registrar")
pestanas.set_title(1, "Consultar")
pestanas.set_title(2, "Informes")


def iniciar_interfaz():
    """Muestra la interfaz interactiva en una celda de Jupyter."""
    display(widgets.VBox([encabezado, pestanas],
layout=widgets.Layout(gap="12px")))


# Ejecutar esta línea en una celda de Jupyter Notebook:
iniciar_interfaz()
```